

AUDITORÍA TÉCNICA INTEGRAL

City2Cruise — Shop&Drop; Port Hub

Plataforma de logística de última milla para cruceristas

Cliente / Entidad	REKER Tech Solutions S.L.
Proyecto	APP_TRASNORTE_LOCKERS_BARCELONA — fase 2-4
Repositorio	github.com/pablete64/APP_TRASNORTE_LOCKERS_BARCELONA
Rama auditada	FASE4-FASE5-FASE6
Versión	0.0.0-development (semantic-release)
Fecha de auditoría	29 de abril de 2026
Auditor	Auditor de software senior — análisis estático + ejecución
Alcance	Frontend, Backend, IaC, CI/CD, seguridad, observabilidad, calidad
Resultado global	APROBADO con observaciones (ver § 11)

1. Resumen ejecutivo

El proyecto **City2Cruise (Shop&Drop; Port Hub)** es una plataforma web/PWA y backend para la recogida y depósito de paquetes en taquillas inteligentes (smart lockers) destinada a cruceristas en puerto. El monorepo contiene un frontend React 18 + Vite + TypeScript con shadcn/ui, un backend Express 5 + Socket.IO + PostgreSQL/PostGIS + Redis, infraestructura como código en Terraform (stack primario Fly.io + Neon + Upstash; alternativa AWS ECS), tres microservicios auxiliares (digital twin Python, rl_service Python para asignación con RL, y un servicio de notificaciones push), CI/CD multi-job en GitHub Actions, y una capa de observabilidad madura (Sentry APM, Prometheus + Grafana, alerting con Alertmanager, logging centralizado con request-id). El nivel de madurez del proyecto es **alto**: Fases 4 (Performance / a11y / i18n), 5 (DevOps / IaC / Observabilidad) y 6 (Tests / Security / Load) figuran completadas según los HITOs documentados.

Veredicto general: el proyecto es **APTO para entrar en producción**, con la salvedad de un conjunto reducido de observaciones de severidad media-alta — la mayoría relacionadas con vulnerabilidades en dependencias del frontend, hardening del contenedor Nginx, y limpieza de secretos por defecto en backend (claves VAPID embebidas en el código). Ninguna de las observaciones es bloqueante en sí misma, pero todas deben resolverse antes de exponer el servicio a tráfico real.

Distribución de hallazgos por severidad

Severidad	Nº	Descripción resumida
CRÍTICO	0	Sin hallazgos bloqueantes
ALTO	3	VAPID hardcoded, vulns axios/lodash, contenedores como root
MEDIO	6	Headers nginx, esbuild/vite/postcss, tsconfig laxo, CORS regex, console.log, hardcoded IP
BAJO	5	Mejoras de hardening, documentación, limpieza de archivos backup
INFO	4	Buenas prácticas extra opcionales

Fortalezas destacadas

- Arquitectura monorepo bien delimitada con frontera explícita frontend/backend/IaC.
- Seguridad backend muy sólida: Helmet con HSTS+CSP+frameguard+noSniff, rate limiter Redis, JWT+refresh con rotación, rotación de claves de cifrado de campo, body-parser con límite 16 KB, CORS allow-list y validación con Zod en schemas.
- **0 vulnerabilidades** en backend (npm audit) sobre 411 dependencias instaladas.
- CI/CD multi-job: lint + tsc + tests + build + audit + secret-scan (Gitleaks) + Docker smoke-build + Digital Twin pytest + ZAP baseline.
- Observabilidad de nivel SaaS: Sentry, Prometheus métricas HTTP/WS/negocio, Grafana, alert-rules, request-id correlation, healthchecks /health (liveness) y /ready (readiness).
- IaC en Terraform para dos stacks alternativos (Fly+Neon+Upstash y AWS ECS) con ADR-001 documentando la decisión.
- Cobertura de tests considerable: **26 tests Jest** en backend (incluido security, audit, concurrency, RBAC, rate-limit, telemetry-pipeline) + **23 tests Vitest** en frontend (incluida suite de a11y con axe-core) + **6 specs Playwright e2e** multi-rol y multi-contexto + tests pytest en digital_twin y rl_service + suite de carga k6.

Debilidades y riesgos clave

- **VAPID keys hardcoded** en *backend/src/config/env.ts*: las claves pública y privada para Web Push tienen valores reales como fallback, en lugar de fallar duramente como JWT_SECRET. Cualquier despliegue sin VAPID_PRIVATE_KEY usará la misma clave privada que cualquier copia del repositorio.
- **7 vulnerabilidades en frontend** (npm audit): 2 high (axios SSRF/DoS, lodash prototype pollution) y 5 moderate (esbuild dev-server, vite, vite-plugin-pwa, follow-redirects, postcss).

- **Contenedores Docker corren como root** (no hay USER en backend/Dockerfile ni frontend/Dockerfile) y no incluyen HEALTHCHECK a nivel de imagen.
- **nginx.conf del frontend no expone cabeceras de seguridad** (HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy). El backend sí las pone vía Helmet, pero la SPA pública la sirve Nginx.
- **tsconfig del frontend con strict deshabilitado** ("strict": false, noImplicitAny:false, strictNullChecks:false). Combinado con 45 ocurrencias de *any* fuera de tests, supone deuda técnica de tipado.
- Archivo *cruise-connect-main/package.json.backup* presente en repo (residuo de migración).

2. Alcance y metodología

La auditoría abarca la rama **FASE4-FASE5-FASE6** del monorepo en su estado actual a 29 de abril de 2026. Se han evaluado los siguientes ejes:

- Arquitectura y stack tecnológico (frontend, backend, IaC, microservicios).
- Seguridad: dependencias (npm audit), secretos, autenticación, autorización, hardening de contenedores y proxy, configuración CSP/CORS, validación de input.
- Calidad de código: tipado TypeScript, configuración ESLint, deuda técnica (any, console.log, TODO/FIXME), modularidad.
- Tests: cobertura unitaria (Jest, Vitest), end-to-end (Playwright), accesibilidad (axe-core), carga (k6) y simulación (digital twin / RL).
- Infraestructura y despliegue: Dockerfiles, docker-compose, Terraform (AWS y Fly+Neon+Upstash), Fly.toml, Nginx, GitHub Actions (CI, CD, e2e, release, ZAP, backup).
- Observabilidad y operabilidad: Sentry, Prometheus, Grafana, alert-rules, logging, healthchecks, runbook DR, gestión de secretos.
- Documentación: README, MEMORIA_TECNICA, ADRs, docs/devops, docs/audits, runbooks.

Metodología combinada: análisis estático de código y configuración + ejecución de herramientas reales (*npm audit* en backend y frontend, intento de *tsc --noEmit* y *eslint*, lectura completa de configuraciones críticas y tests). Se han clasificado los hallazgos según el modelo CVSS-like de severidad (Crítico / Alto / Medio / Bajo / Info) atendiendo a impacto, explotabilidad y exposición.

3. Inventario del proyecto

Área	Métrica	Valor
Tamaño	Ficheros TS/TSX/JS (sin node_modules)	243
Tamaño	Ficheros .md	44
Tamaño	Ficheros Terraform (.tf)	14
Frontend	Líneas TS/TSX en src/	11.926
Frontend	Tests Vitest (.test.* / .spec.*)	23
Frontend	Specs E2E Playwright	6 (admin, client, driver, smoke, multi-context, visual-reg)
Backend	Líneas TS en src/	11.030
Backend	Tests Jest	26 (security, RBAC, audit, concurrency, telemetry, rate-limiter...)
Backend	Rutas API expuestas	14 routers (auth, cruises, lockers, payments, push, requests, ...)
Microservicios	digital_twin (Python)	Tests pytest, integración con backend y rl_service
Microservicios	rl_service (Python)	Twin bridge tests
IaC	Stack primario	Fly.io + Neon (Postgres+PostGIS) + Upstash Redis
IaC	Stack alternativo	AWS VPC + ALB + ECS Fargate + RDS + ElastiCache + CloudFront
CI/CD	Workflows GitHub Actions	8 (ci, cd, cd-frontend, e2e, release, commitlint, zap, backup)
Documentación	ADR + HITOs DevOps + runbooks	ADR-001 + 21 HITOs DevOps + DR_RUNBOOK

4. Arquitectura y stack tecnológico

4.1 Frontend

React 18.3 + Vite 5.4 + TypeScript 5.8 con Tailwind 3.4 y shadcn/ui (Radix). Routing con react-router-dom v6. Tiempo real con socket.io-client. Mapas con react-leaflet + Leaflet marker-cluster. Estado de servidor con @tanstack/react-query v5. Validación con Zod + react-hook-form + @hookform/resolvers. PWA con vite-plugin-pwa (Workbox). Sentry SDK para errores en producción. Integración Stripe Elements (@stripe/react-stripe-js). i18n con i18next + react-i18next (5 idiomas).

4.2 Backend

Express 5 (importante: usa la última major) sobre Node 20 LTS. PostgreSQL (Neon en producción) con PostGIS para queries geoespaciales y cursor pagination keyset. Redis (Upstash) para cache, rate-limit compartido entre workers y socket.io-redis-adapter. JWT HS256 con refresh-token y passwordPolicy. Helmet + CORS allow-list. Rate-limit global + específicos (auth, lockerOpen, handshake). Webhook Stripe con body raw. Push notifications con web-push (VAPID). Twilio SMS. Sentry (instrumentación temprana, OTel). Prometheus prom-client. Pino logger. Cluster mode (node:cluster) opcional, alternativa PM2 con ecosystem.config.cjs.

4.3 Microservicios y twin

- **digital_twin** (Python): simulador de eventos para validación de la lógica de matching y telemetría — integrado en CI con pytest.
- **rl_service** (Python): microservicio de Reinforcement Learning para ranking de drivers (PPO). Activable con RL_ROUTING_ENABLED=true; fallback geo-distance si timeout.
- Comunicación interna autenticada con shared secret (X-Internal-Key + INTERNAL_API_KEY).

4.4 Infraestructura

- **Stack primario:** Fly.io (machines en MAD), Neon Postgres serverless con PITR, Upstash Redis serverless con TLS, Cloudflare Pages para el frontend, Sentry, Grafana Cloud.
- **Stack alternativo:** AWS VPC + ALB + ECS Fargate + RDS + ElastiCache + CloudFront — preparado en terraform/aws/ por si se supera el free tier.
- **Tres entornos** (dev, staging, production) con .env.example + workflow de gestión de secretos documentado en HITO_5_2_5.

5. Hallazgos de seguridad

#S-01 · Claves VAPID por defecto hardcodeadas en el código

ALTO

Evidencia: *backend/src/config/env.ts* líneas 71-74: `publicKey: process.env.VAPID_PUBLIC_KEY || 'BGscsMy01ynE...ZQsE'; privateKey: process.env.VAPID_PRIVATE_KEY || '0XNrTZGcD0...vftdAis'`. A diferencia de `JWT_SECRET`, `REFRESH_TOKEN_SECRET`, `FIELD_ENCRYPTION_KEY`, `STRIPE_SECRET_KEY` e `INTERNAL_API_KEY` (todas con guard que lanza Error en producción), las VAPID keys aceptan en silencio el fallback hardcoded.

Riesgo: Una clave privada VAPID expuesta permite a un atacante firmar notificaciones push falsas para los suscriptores. Si la app despliega con la variable sin definir, todos los usuarios reciben push firmadas con la clave pública del repo — y cualquier copia del repo puede emitir notificaciones legítimas.

Recomendación: Aplicar el mismo patrón que el resto: `if (!key && NODE_ENV==='production') throw new Error('FATAL: VAPID_PRIVATE_KEY obligatorio')`. Rotar la clave actual y eliminar el literal del código (quedará solo en `envs/production.env` real, fuera de git).

#S-02 · Vulnerabilidades altas en dependencias del frontend (axios, lodash)

ALTO

Evidencia: *cruise-connect-main* · npm audit reporta 7 advisories — 2 HIGH y 5 MODERATE: axios <=1.14.0 (SSRF GHSA-jr5f-v2jv-69x6, DoS GHSA-4hjh-wcwx-xvwj, prototype pollution GHSA-43fc-jf86-j433, NO_PROXY bypass GHSA-3p68-rc4w-qgx5, header injection GHSA-fvcv-3m26-pcqx); lodash <=4.17.23 (prototype pollution GHSA-xxjr-mmjv-4gpg, code injection GHSA-r5fr-rjxr-66jc).

Riesgo: axios se usa en el cliente HTTP del frontend (*services/api.ts*) y los advisories afectan a casos donde un usuario puede influir en la URL de destino o headers; lodash es transitiva y la prototype pollution puede ser explotada vía objetos no confiables. Aunque la política documentada del CI permite high como warning, son vulnerabilidades reales con CVE público.

Recomendación: Ejecutar `npm audit fix` en *cruise-connect-main*/. Si quedan transitivas, añadir overrides en `package.json` (ya hay 3 overrides definidos). Plazo recomendado: <15 días.

#S-03 · Contenedores Docker corren como usuario root

ALTO

Evidencia: Ni *backend/Dockerfile* ni *cruise-connect-main/Dockerfile* incluyen instrucción `USER`. La capa runtime de Node se ejecuta como UID 0, y la imagen Nginx también queda como root. No hay `HEALTHCHECK` a nivel de imagen (solo en `docker-compose`).

Riesgo: Una RCE en cualquiera de los procesos contenedorizados se ejecutaría con privilegios root dentro del contenedor, facilitando escapes a través de mounts o capabilities y ampliando el blast radius si Fly.io o el orquestador permiten alguna escalada.

Recomendación: Backend: añadir `USER node` tras el `COPY` del runtime stage (node:20-alpine ya incluye el usuario node). Frontend: crear usuario nginx no privilegiado y dar permisos a `/var/cache/nginx + /var/run`. Añadir `HEALTHCHECK` explícito en el `Dockerfile`.

#S-04 · Cabeceras de seguridad ausentes en Nginx (frontend)

MEDIO

Evidencia: *cruise-connect-main/nginx.conf* sirve la SPA al usuario final pero no añade Strict-Transport-Security, Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Referrer-Policy ni Permissions-Policy. El backend sí las añade vía Helmet, pero el primer contacto con el navegador (HTML, JS, assets) viene del Nginx.

Riesgo: Sin CSP, los XSS reflejados/almacenados son de explotación más simple. Sin HSTS un MITM en primera visita puede degradar a HTTP. Sin X-Frame-Options la app puede ser embebida en iframes de origen ajeno (clickjacking).

Recomendación: Añadir bloque `add_header` en *nginx.conf* con HSTS (`max-age 31536000 includeSubDomains preload`), CSP coherente con la app (`script-src 'self', img-src 'self' data: tiles.providers, connect-src 'self' API y socket`), X-Frame-Options DENY, X-Content-Type-Options nosniff, Referrer-Policy strict-origin-when-cross-origin, Permissions-Policy para deshabilitar features no usadas.

#S-05 · Cadena moderate-severity esbuild → vite → vite-plugin-pwa

MEDIO

Evidencia: GHSA-67mh-4wv8-2f99: el dev-server de esbuild ($\leq 0.24.2$) acepta peticiones cross-origin y permite leer respuestas. Afecta solo a entornos de desarrollo, pero está en la cadena del proyecto vía vite (overridden a 5.4.21 que arrastra esbuild vulnerable) y vite-plugin-pwa.

Riesgo: En desarrollo local (`npm run dev`) cualquier sitio puede leer respuestas del dev server. No afecta al build de producción, pero un dev con tokens en localStorage podría exponerlos a un navegador con varias pestañas abiertas.

Recomendación: Actualizar a vite ≥ 6 / esbuild $> 0.24.2$ cuando vite-plugin-pwa publique versión compatible. Mientras tanto, evitar abrir pestañas no confiables durante el desarrollo y considerar `--host 127.0.0.1` en vez de 0.0.0.0.

#S-06 · Política CORS con regex permisivo en desarrollo

MEDIO

Evidencia: *backend/src/server.ts*: la lista CORS añade `origin.startsWith('http://localhost:')` y, si no es producción, `origin.startsWith('http://192.168.')`. En *sockets/io.ts* el patrón es idéntico. *credentials: true* está activo.

Riesgo: Aceptar cualquier `http://localhost:*` con `credentials=true` es razonable en dev pero amplía la superficie si por error se despliega `NODE_ENV  $\neq$  'production'` (p.ej. 'development' en staging). En producción solo entra el origen exacto del frontendUrl, lo cual es correcto, pero conviene reforzarlo.

Recomendación: 1) Asegurar que `NODE_ENV='production'` es invariante en staging y prod (verificable en *fly.production.toml* ya lo hace). 2) Sustituir `startsWith` por regex anclado y con whitelisting explícito de puertos esperados. 3) Loggear y alertar (Sentry) cualquier rechazo CORS para detectar configuración errónea.

#S-07 · IP local de desarrollo hardcodeda en el log de arranque

MEDIO

Evidencia: *backend/src/index.ts*: `logger.info({ url: `http://192.168.1.47:${config.port}/api/health` })`. La IP 192.168.1.47 corresponde al equipo de un desarrollador y se imprime en cada arranque, incluido producción.

Riesgo: Exposición menor de información interna en logs centralizados. No es explotable por sí misma pero ensucia trazas y revela layout de red del equipo.

Recomendación: Sustituir por la IP/host real del proceso (`os.hostname()` o `config.frontendUrl`) o dejar solo el puerto.

#S-08 · Logs con console.log fuera del logger estructurado

BAJO

Evidencia: 39 ocurrencias de `console.log` en `src/` (excluyendo tests). El proyecto usa pino (backend) y el frontend tiene Sentry. Cualquier `console.log` se escapa del pipeline de logging (no lleva request-id, no se filtra por nivel, no llega a Loki/Better Stack).

Riesgo: Pérdida de correlation y posible fuga de información sensible (objetos completos) en stdout que llega al sistema de logs como cadenas no estructuradas.

Recomendación: Reemplazar por `logger.info/.debug/.warn` o eliminar. Activar regla ESLint `no-console` con `allow-list` para `warn/error`.

#S-09 · Archivo de backup obsoleto en el repositorio

BAJO

Evidencia: `cruise-connect-main/package.json.backup` (residuo de migración). También existe `fix_vulnerabilities.sh` y `start_all.sh` dentro del frontend.

Riesgo: `package.json.backup` confunde a auditores (puede contener dependencias antiguas con CVEs ya corregidos en el real) y sigue siendo indexado por GitHub. Los `.sh` sueltos dentro del frontend mezclan lógica de orquestación con código de aplicación.

Recomendación: Eliminar `package.json.backup`. Mover los scripts `.sh` a `/scripts/` con prefijo del entorno.

#S-10 · Configuración estricta de TypeScript desactivada en frontend

MEDIO

Evidencia: `cruise-connect-main/tsconfig.app.json`: `strict:false`, `noImplicitAny:false`, `noUnusedLocals:false`, `noUnusedParameters:false`, `noFallthroughCasesInSwitch:false`. `tsconfig.json` raíz también pone `strictNullChecks:false`. 45 ocurrencias de `any` (o `as any`) en `src/` fuera de tests.

Riesgo: Pérdida de garantías de tipado en una codebase de 11.926 líneas. Mayor probabilidad de bugs tipo "cannot read property of undefined" llegando a producción y dificulta refactors futuros con confianza.

Recomendación: Plan progresivo: activar `strictNullChecks` y `noImplicitAny` en módulos nuevos primero (vía `// @ts-strict` o `tsconfig` por carpeta) y migrar el resto en sprints. Habilitar la regla `typescript-eslint/no-explicit-any` en `eslint.config.js`.

6. Calidad de código y tests

6.1 Linting y tipado

ESLint v9 con flat-config en frontend (typescript-eslint, react-hooks, react-refresh). El backend no tiene ESLint dedicado (solo el del monorepo), aunque *strict:true* en tsconfig.json compensa parcialmente. Recomendación: añadir ESLint al backend con preset *@typescript-eslint/recommended-type-checked* y *no-floating-promises* activado.

6.2 Cobertura de tests

- **Backend** — 26 tests Jest cubren rutas críticas (auth, RBAC, payments-flow, audit, concurrency con SIMULATE_RACE, rate-limiter Redis, cluster, cache, cursor-pagination, compression+ETag, metrics, health, twin-sync, telemetry-pipeline, chain-of-custody, merchants, security, handshake-ratelimit). Hay un test *renew-handshake* que ejercita el flujo de renovación.
- **Frontend** — 23 tests Vitest, incluyendo carpeta dedicada `__tests__/a11y/` con axe-core para WCAG AA. Existe variante *test:a11y:strict*.
- **E2E** — 6 specs Playwright: smoke, client-flows, driver-flows, admin-flows, multi-context-realtime (sockets en paralelo), visual-regression. Workflow e2e.yml dedicado.
- **Carga** — k6 con phase4-100c.js y phase4-spike.js, runner scripts/k6-phase4.sh.
- **Microservicios** — pytest en digital_twin/tests y rl_service/tests/test_twin_bridge.py.

6.3 Deuda técnica detectada

- Solo 1 marcador TODO/FIXME/HACK en todo el código de producción — excelente.
- 0 patrones `.then()` sin `.catch()` detectados — bien.
- 1 ocurrencia de `@ts-ignore/@ts-expect-error` — aceptable.
- 39 `console.log` a migrar a logger (S-08).
- 45 usos de *any* (S-10).
- tsconfig laxo en frontend (S-10).
- **No se detectaron patrones de concatenación SQL** — todas las queries usan placeholders \$1, \$2... con pg.

7. Infraestructura y despliegue

7.1 Docker y docker-compose

Dockerfiles multi-stage correctos (builder + runtime). docker-compose.yml para producción local (db + backend + frontend con healthchecks) y docker-compose.dev.yml con Redis añadido y volúmenes bind-mount para hot-reload. Mejoras: USER no-root (S-03), HEALTHCHECK a nivel imagen, copiado selectivo (.dockerignore correcto en frontend).

7.2 Terraform

Dos módulos completos:

- **terraform/flyneonupstash/** — providers Neon + Upstash + Fly + Cloudflare, rama por entorno en Neon (PITR), endpoint read_write, fly_volume persistente, fly_machine para 2 réplicas en MAD, output del DATABASE_URL, REDIS_URL, etc.
 - **terraform/aws/** — VPC + subnets pub/priv en N AZs, IGW, security-group ALB, ECS Fargate, RDS, ElastiCache, CloudFront. Listo como plan B.
- .gitignore de terraform correcto (excluye .tfstate*, *.tfvars salvo *.example).

7.3 Fly.io producción

fly.production.toml: 2 machines mínimas (HA), shared-cpu-2x / 512 MB, force_https, auto_stop_machines=false, healthcheck cada 10s a /api/health, hard_limit 500 conexiones, kill_timeout 30s para graceful shutdown. Adecuado para arranque de producción; revisar memory_mb=512 si k6 muestra OOM bajo carga.

7.4 CI/CD

8 workflows en .github/workflows: **ci.yml** (frontend lint+tsc+vitest+build, backend tsc+jest+build con servicios Postgres+Redis levantados, security npm audit + Gitleaks, docker smoke-build, digital-twin pytest, rl-service bridge), **cd.yml** y **cd-frontend.yml** (build+push GHCR con tags semver+sha+latest), **e2e.yml**, **release.yml** con semantic-release, **commitlint.yml** con conventional, **zap-baseline.yml** y **backup.yml**. Permisos minimizados (contents:read por defecto). Excelente.

Observación: el job security usa `npm audit --audit-level=high || echo warning`. Política documentada (Hito 6.3.2): critical bloquea, high warning. Coherente pero implica vivir con HIGH abiertas hasta los 30 días — al ritmo actual conviene cerrar S-02.

8. Observabilidad y operabilidad

- **Sentry:** SDK v8 en backend (instrumentado antes de cualquier import) y v8 en frontend. Sample rates parametrizados por entorno (10% prod), ignoreErrors razonable, release vinculado a npm_package_version.
- **Prometheus:** *prom-client* con métricas HTTP (*httpMetricsMiddleware*), websockets (*wsConnections*, *driversOnline*) y de negocio. Endpoint */metrics* expuesto. Scrape config en *observability/prometheus.scrape.yml*.
- **Grafana + Alertmanager:** dashboards en *observability/grafana/*, *alert-rules.yml* con reglas de SLO y *alertmanager.yml* configurado.
- **Healthchecks** separados (Hito 5.3.6): */health* (liveness, sin chequeos externos) y */ready* (readiness con DB + Redis). Diseño correcto para Fly y k8s.
- **Request-id:** middleware antes de healthchecks y rate-limit, todos los logs Pino incluyen el id (*X-Request-Id*) para correlation cross-stack.
- **Runbook DR:** *docs/runbooks/DR_RUNBOOK.md* + workflow *backup.yml* + scripts/backup/.
- **ADRs:** ADR-001 documenta la elección Fly+Neon+Upstash y descarta AWS por coste/free tier.

9. Documentación

Es un punto fuerte del proyecto. Conviven 44 ficheros .md, distribuidos así:

- **Memoria técnica y auditorías previas** (incluyendo AUDITORIA_TECNICA_CITY2CRUISE v1, RE_AUDITORIA v2, RESPUESTA_CONSULTAS_TECNICAS, varios PLAN_EJECUCION).
- **docs/devops/:** 21 HITOs documentados (5.1.3 Branch Protection, 5.2.2 Terraform, 5.2.3 Three Environments, 5.2.4 Backups & DR, 5.2.5 Secrets Management, 5.3.* Sentry/Prometheus/KPIs/Alerting/Logging/Healthchecks, 5.4.* Digital Twin/Sim-to-Real/Twin Telemetry/Control Tower, 6.2/6.3/6.4/6.5 Tests/Security/Load/AI Validation + RESÚMENES finales).
- **docs/audits/:** HITO_4_1_AUDITORIA_A11Y, HITO_4_3_3_SQL, HITO_4_3_5_K6, HITO_4_RESUMEN_FINAL, a11y-results.json, bundle/.
- **docs/adrl:** ADR-001-cloud-provider.
- **docs/runbooks/:** DR_RUNBOOK.
- **docs/PHASE3_CLOSURE.md.**

Recomendación: consolidar las múltiples auditorías y planes de ejecución en una carpeta *docs/history/* y dejar en raíz solo README + MEMORIA_TECNICA + esta auditoría 2026-04-29.

10. Tabla consolidada de hallazgos

ID	Severidad	Área	Hallazgo	Esfuerzo
S-01	ALTO	Seguridad / config	VAPID keys hardcodeadas como fallback	1 h
S-02	ALTO	Seguridad / deps	axios y lodash con CVEs HIGH (frontend)	1-3 h
S-03	ALTO	Seguridad / Docker	Contenedores corren como root, sin HEALTHCHECK	2 h
S-04	MEDIO	Seguridad / Nginx	Headers de seguridad ausentes en frontend	1 h
S-05	MEDIO	Seguridad / dev	esbuild dev-server CORS issue (cadena vite)	0 h*
S-06	MEDIO	Seguridad / CORS	Regex permisivo en dev/staging	2 h
S-07	MEDIO	Logs / IP	IP local hardcodeada en log de arranque	10 min
S-08	BAJO	Calidad / logging	39 console.log fuera del logger estructurado	2 h
S-09	BAJO	Repo / cleanup	package.json.backup y scripts sueltos en repo	10 min
S-10	MEDIO	Calidad / tipado	tsconfig laxo + 45 usos de any en frontend	Iterativo
I-01	INFO	Backend	Añadir ESLint dedicado al backend con no-floating-promises	2 h
I-02	INFO	CI	Considerar SBOM (cyclonedx) y firma de imágenes (cosign)	4 h
I-03	INFO	Docs	Consolidar auditorías históricas en docs/history/	30 min
I-04	INFO	Operación	Verificar memory_mb=512 en Fly bajo carga real	1 h

(*) S-05 no requiere esfuerzo inmediato; queda a la espera de release de vite-plugin-pwa compatible con vite ≥6.

11. Plan de remediación recomendado

11.1 Bloque inmediato (antes de exponer a tráfico real, ~ 1 jornada)

- **S-01 VAPID keys:** aplicar guard `NODE_ENV='production'`, rotar la clave, eliminar literales del código.
- **S-02 npm audit:** ejecutar `npm audit fix` en frontend; añadir overrides si quedan transitivas.
- **S-03 Docker non-root:** añadir USER node (backend) y un usuario no privilegiado en el stage Nginx + HEALTHCHECK por imagen.
- **S-04 Headers Nginx:** añadir bloque `add_header` con HSTS, CSP, X-Frame-Options, etc.
- **S-07 IP hardcoded:** limpiar el log de arranque.

11.2 Bloque corto plazo (semana siguiente, ~ 2-3 jornadas)

- S-06 reforzar CORS con regex anclado y alertas.
- S-08 migrar console.log a logger; activar regla ESLint no-console.
- S-09 limpiar repo (package.json.backup, scripts sueltos).
- S-10 plan progresivo de strictness en frontend (módulos nuevos en strict).
- I-01 ESLint en backend con no-floating-promises.

11.3 Bloque medio plazo (siguiente sprint, ~ 1 semana)

- I-02 generar SBOM cyclonedx y firmar imágenes con cosign.
- I-03 reorganizar docs/ en docs/history.
- I-04 ejecución real de k6 contra staging para validar memory_mb.
- Cerrar S-05 en cuanto vite-plugin-pwa publique versión compatible.

12. Conclusión

El proyecto City2Cruise se encuentra en un estado de madurez técnica notable. La separación de responsabilidades es clara, el stack está alineado con las mejores prácticas (Express 5, React 18, TypeScript, PostGIS, Redis cluster, Sentry, Prometheus, Terraform, semantic-release, conventional-commits), la cobertura de tests es real y multi-nivel (unit + integración + e2e + carga + a11y), y la documentación de cada hito DevOps es de alto nivel.

No se ha identificado ningún hallazgo crítico bloqueante. Las observaciones de severidad **ALTA** son resolubles en una jornada de trabajo y se concentran en (a) sustituir las claves VAPID por defecto por un guard que falle en producción, (b) cerrar las vulnerabilidades públicas de axios y lodash en el frontend y (c) endurecer el contenedor (USER no-root y HEALTHCHECK). Tras esas correcciones, el proyecto puede considerarse **listo para producción controlada** (canary/staging real con tráfico orgánico) y un cierre formal de Fase 6.

Recomendación final: ejecutar el bloque inmediato del plan §11.1 antes de la primera exposición pública, reservar el bloque corto plazo §11.2 para la semana de hardening post-launch y el bloque medio plazo §11.3 para el siguiente sprint, junto con un re-audit ligero (npm audit + zap baseline + revisión de Sentry) a 30 días.

— Fin del informe —